Singly Linked List – Insert At End And Time Complexity.

Program:

```
#include <iostream>
#include <stack>
using namespace std;

typedef struct ListNode
{
    int data;
    struct ListNode *next;
} Node;

Node *head;

void createEmptyList(Node **head)
{
    *head = nullptr;
    cout << "Empty list created!!!" << endl;
}

void insertAtBeginning(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    if (*head == nullptr)
    {
        newNode->next = nullptr;
        *head = newNode;
    }
```

```
    else
    {
        newNode->next = *head;
        *head = newNode;
    }
}
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
    newNode->data = data;
    newNode->next = nullptr;
    if (*head == nullptr)
    {
        *head = newNode;
    }
    else
    {
        Node *current = *head;
        while (current->next != nullptr)
        {
            current = current->next;
        }
        current->next = newNode;
    }
}


int main()
{
   createEmptyList(&head);
    while (1)
    {
```

```
        cout << "1. Insert at the beginning" << endl;
        cout << "2. Insert at the end" << endl;

        int choice;
        cout << "Enter your choice: ";
        cin >> choice;
        switch (choice)
        {
        case 1:
        {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtBeginning(&head, data);
            break;
        }
      case 2:
      {
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
            break;
      }
       default:
        {
            cout << "Invalid choice." << endl;
            break;
        }
        }
    }
  return 0;
}
```

Program Analysis:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
```

→ *The size of newNode will be equal to the size of a Node structure and newNode is declared as a pointer to Node structure.*

→ *The size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

→ *The size of the Node structure itself is determined by the sum of the sizes of its members*:

→ *data(an integer) is typically 4 bytes.*

→ *next(a pointer to another Node) is typically 8 bytes.*

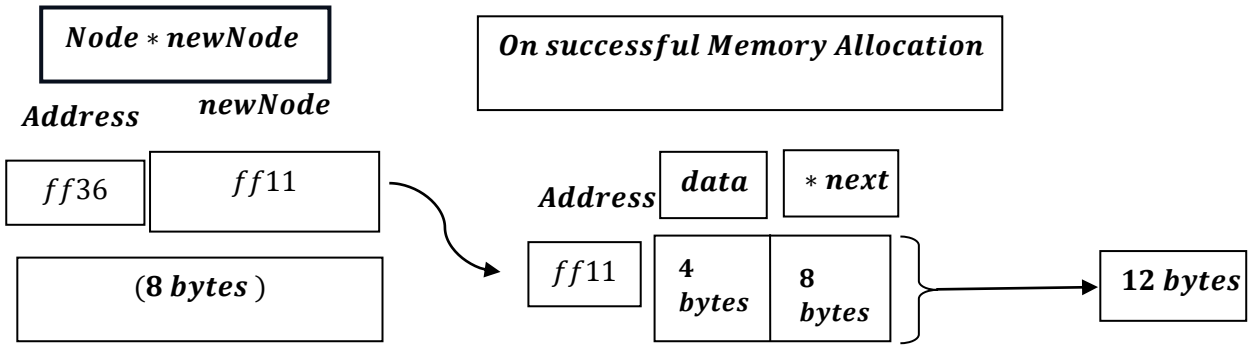
→ *Therefore , the size of the Node structure is typically 12 bytes(4 bytes for data + 8 bytes for next) .*

```
    if (newNode == nullptr)
    {
        cout << "Error: Memory allocation failed." << endl;
        return;
    }
```

What does it mean?

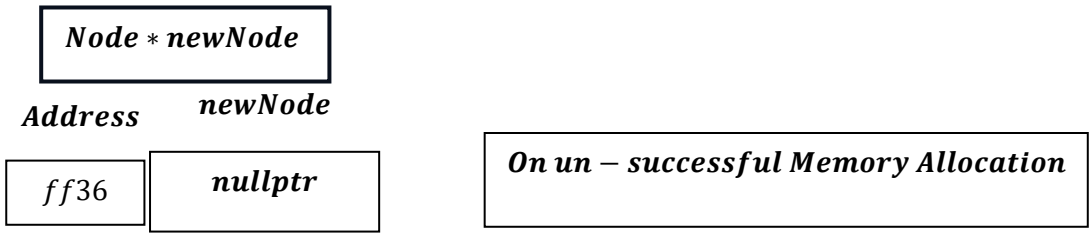
→ *newNode will try to allocate memory for data and next pointer variable i.e. typically 8 bytes, though for variable data and structure Node's pointer variable next in total 12 bytes, but as discussed earlier , the size of a pointer (newNode) is typically 8 bytes on most modern 64 − bit systems, regardless of the size of the structure it points to.*

Say,



But on Un − Successful memory allocation:

→ ***if the allocation fails due to insufficient memory or other reasons, malloc will return a null pointer***.



Now, if the value of the pointer variable newNode is nullptr, then:

One may output : `Overflow ***or Memory Allocation Failed as a message***.`

Now, there are two condition before us :

→ ***1st condition we have just created an empty list and Node * head set to Null and we want to insert at end of linked list***.

→ ***2nd Condition we have is head is pointed to linked nodes contain list of data and address and head is not pointed to null.***

1st condition :

$Node * head$	
Address	***Head***
$ff21$	***nullptr***

Then:

```
case 2:
{
        int data;
        cout << "Enter the data: ";
        cin >> data;
        insertAtEnd(&head, data);
        break;
}
```

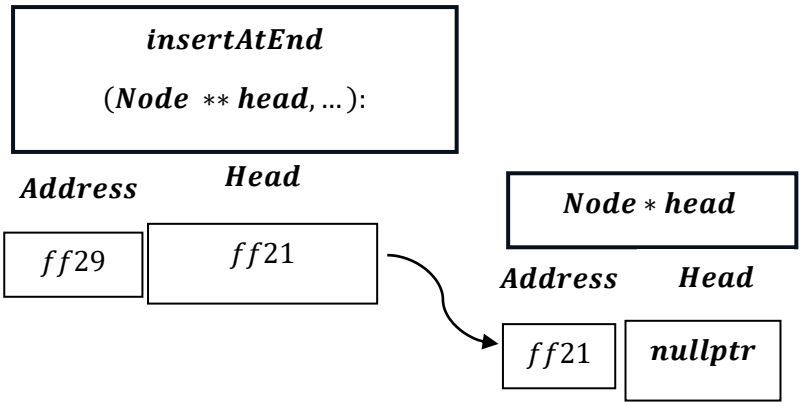
Say, data is 12 , hence we pass through function call:

$$insertAtEnd(ff21, 12)\ ;$$

Now,

```
void insertAtEnd(Node **head, int data) {…}
```

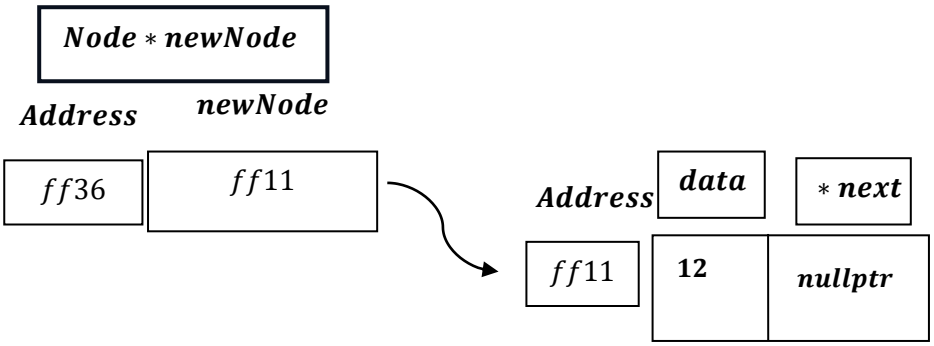
This creates: –



Now,

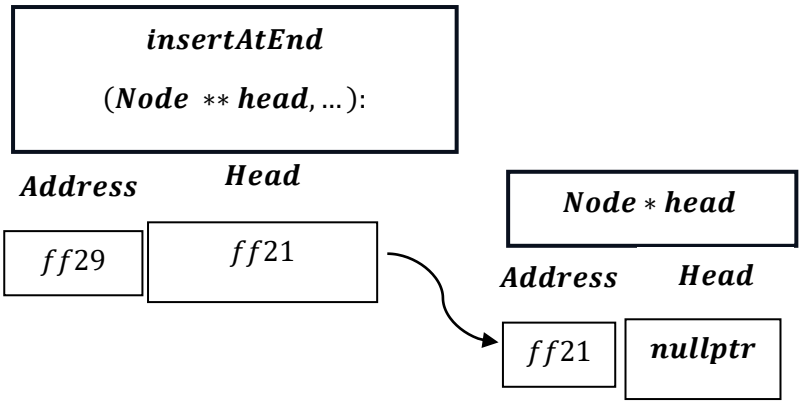
```
    newNode->data = data;
    newNode->next = nullptr;
```

This , initializes , structure Node's member integer variable data with data here it is: 12 *and next pointer member of structure Node to nullptr*.



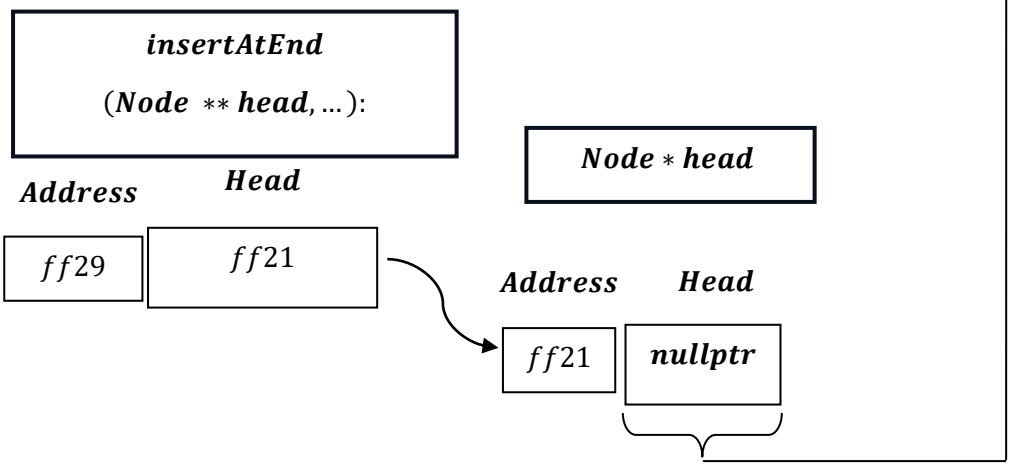
Next,

```
    if (*head == nullptr)
    {
        *head = newNode;
    }
```



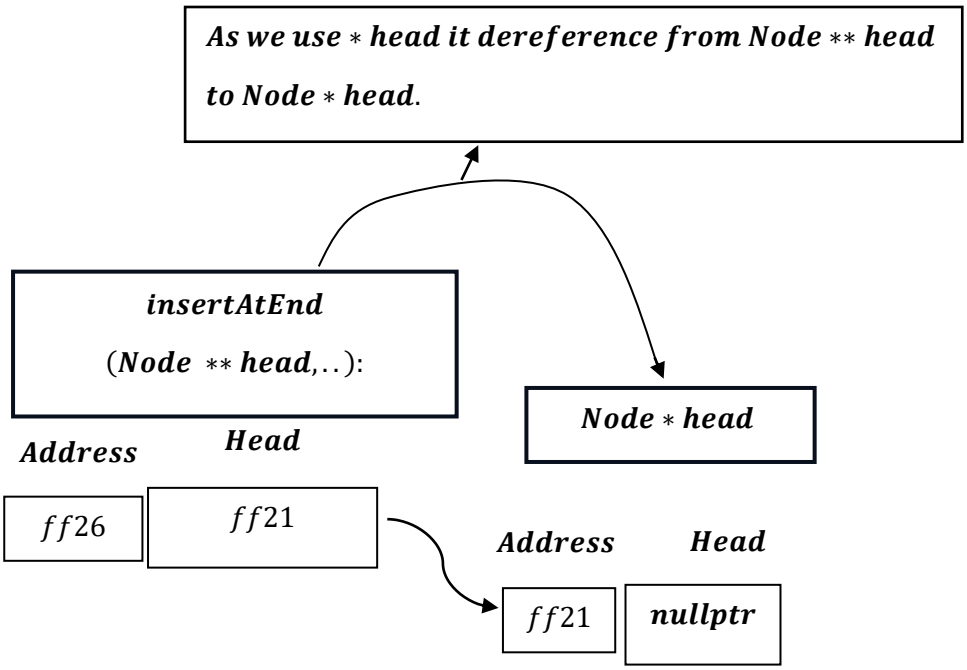
$if(*head == nullptr)$ *which is true*

Why true ?
Ans: Though $*head$ *which is of Node* $**head$ *temporary pointer to pointer variable of the function , as* $*head$ *is used , it dereference from Node* $**head$ *and refer to Node* $*head$ *global pointer to which Node* $**head$ *is pointing to and then its value is checked is whether equal to nullptr or not ? This occurs during execution and compilation of the insertAtEnd function.*

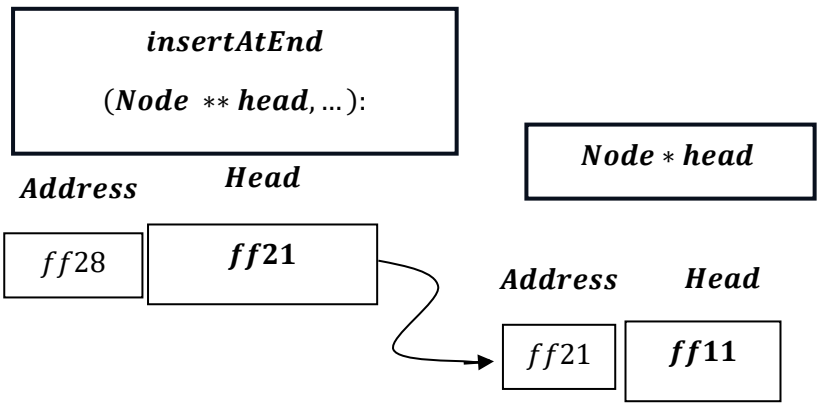


And, $*head = newNode;$

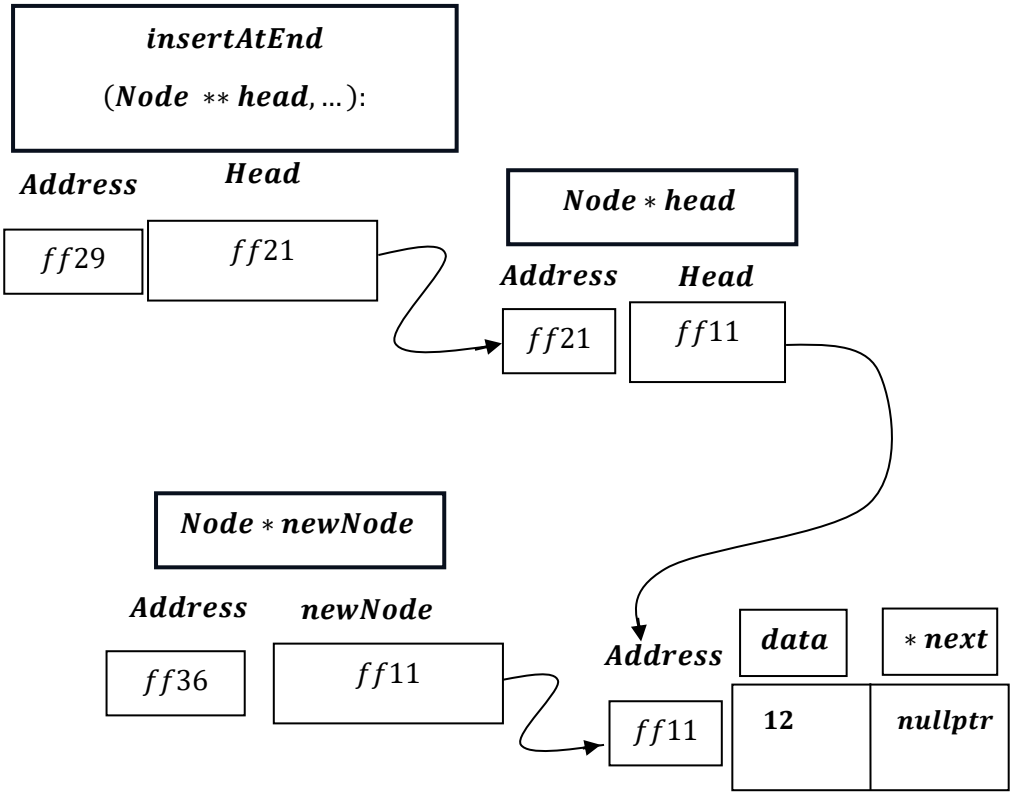
Note as we are using ,* head, where only head refers to Node ** head which is local variable used in the function insertAtEnd. Using a pointer to head i.e.* head , will dereference from Node ** head and refer us to Node * head to which Node ** head points to i.e.



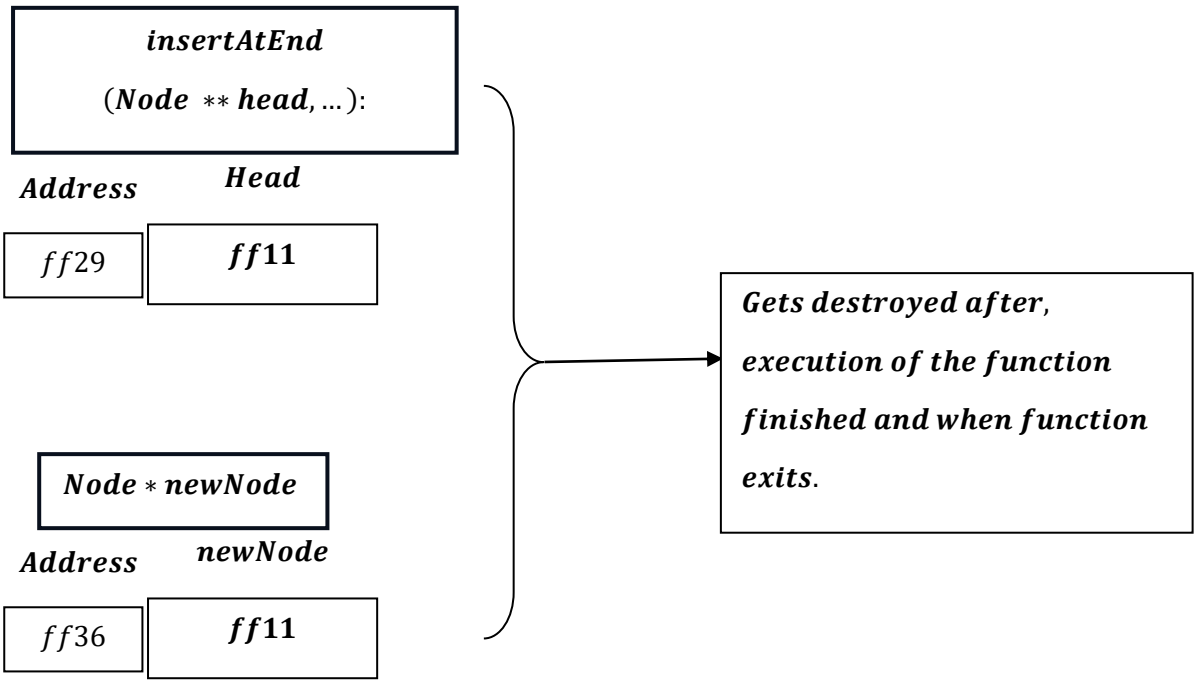
And the value i.e. the address of Node * head[here, ff21] stored in Node ** head will remain unchanged and unaltered , similarly the line of code ` * head = newNode` assigns/changes the value of global pointer Node * head to newNode's value(here , ff11) i.e. address to which newNode pointer pointing , and this happens due to dereferencing of Node ** head to Node * head, as shown below:



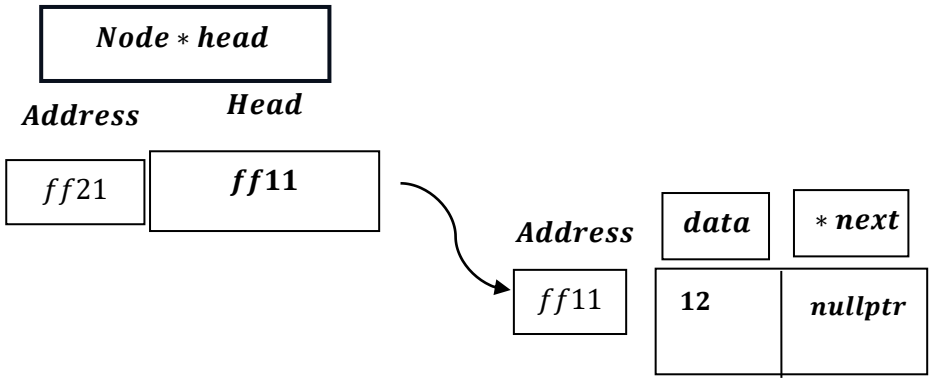
And , then Node * head starts pointing to newNode's value i.e. the address to which newNode pointer pointing to (here, ff11) .



Now, as Node * newNode and Node ** head are temporary variable, therefore gets destroyed after function execution get finished , i.e.

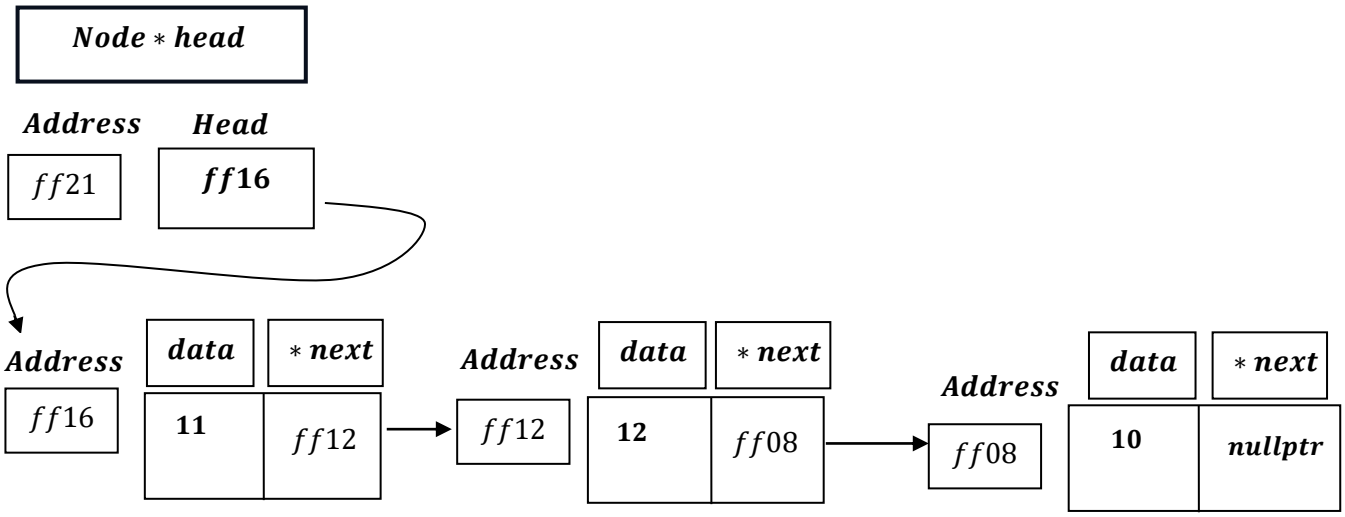


Then we are left with:



2nd condition :

Suppose we have linked list:



And we recall the previous passing by reference of head and value of data.

```
        case 2:
        {
```

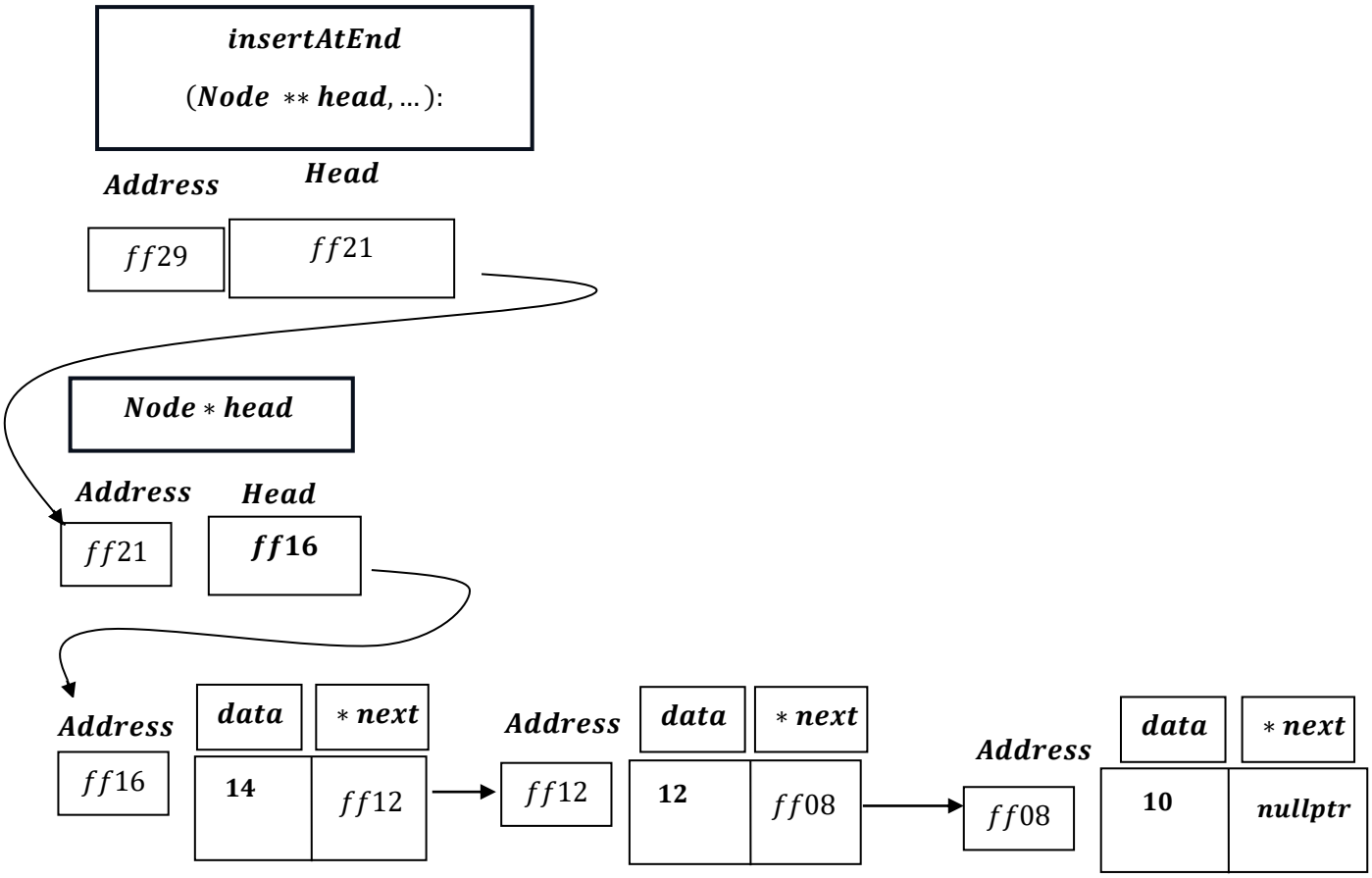
```
            int data;
            cout << "Enter the data: ";
            cin >> data;
            insertAtEnd(&head, data);
            break;
        }
```

$i.e., insertAtEnd(ff21, 8)$;

Now,

```
void insertAtEnd(Node **head, int data) {…}
```

Hence ,

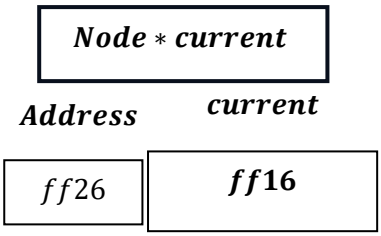


Now,

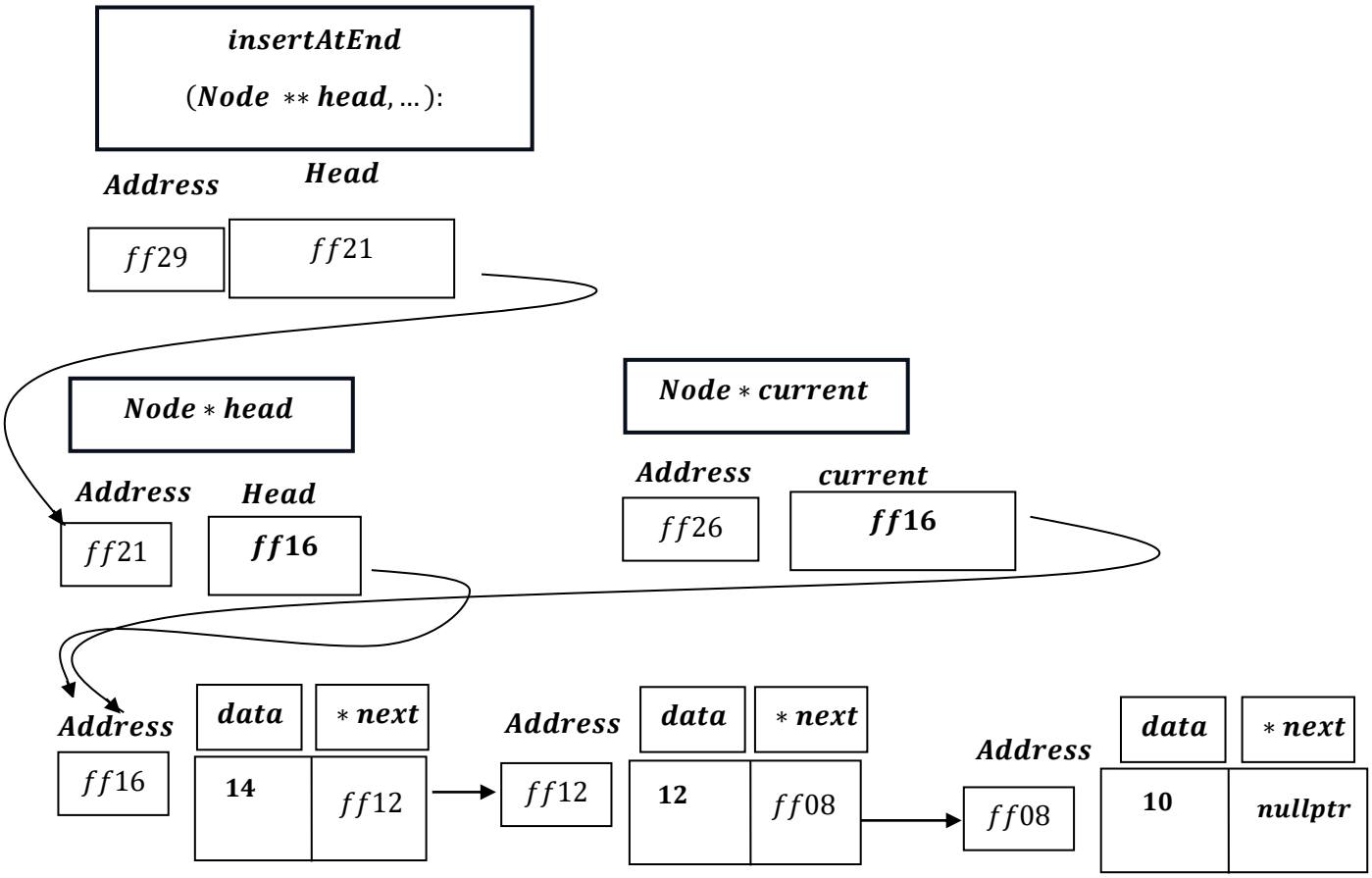
```
else
    {
        Node *current = *head;
        ……….
```

```
    }
}
```

current is a pointer of Node structure which assigned to the value of pointer head.



Why ff16 ? , when we are using * head, i.e. here head used is of local pointer to pointer i.e. Node ** head , and as * head is used, it will first derefer from Node ** head and refer us to Node * head , which will cause current pointer to store value contained in Node * head i.e. ff16 , address of Head node to which Node * head pointer points to.



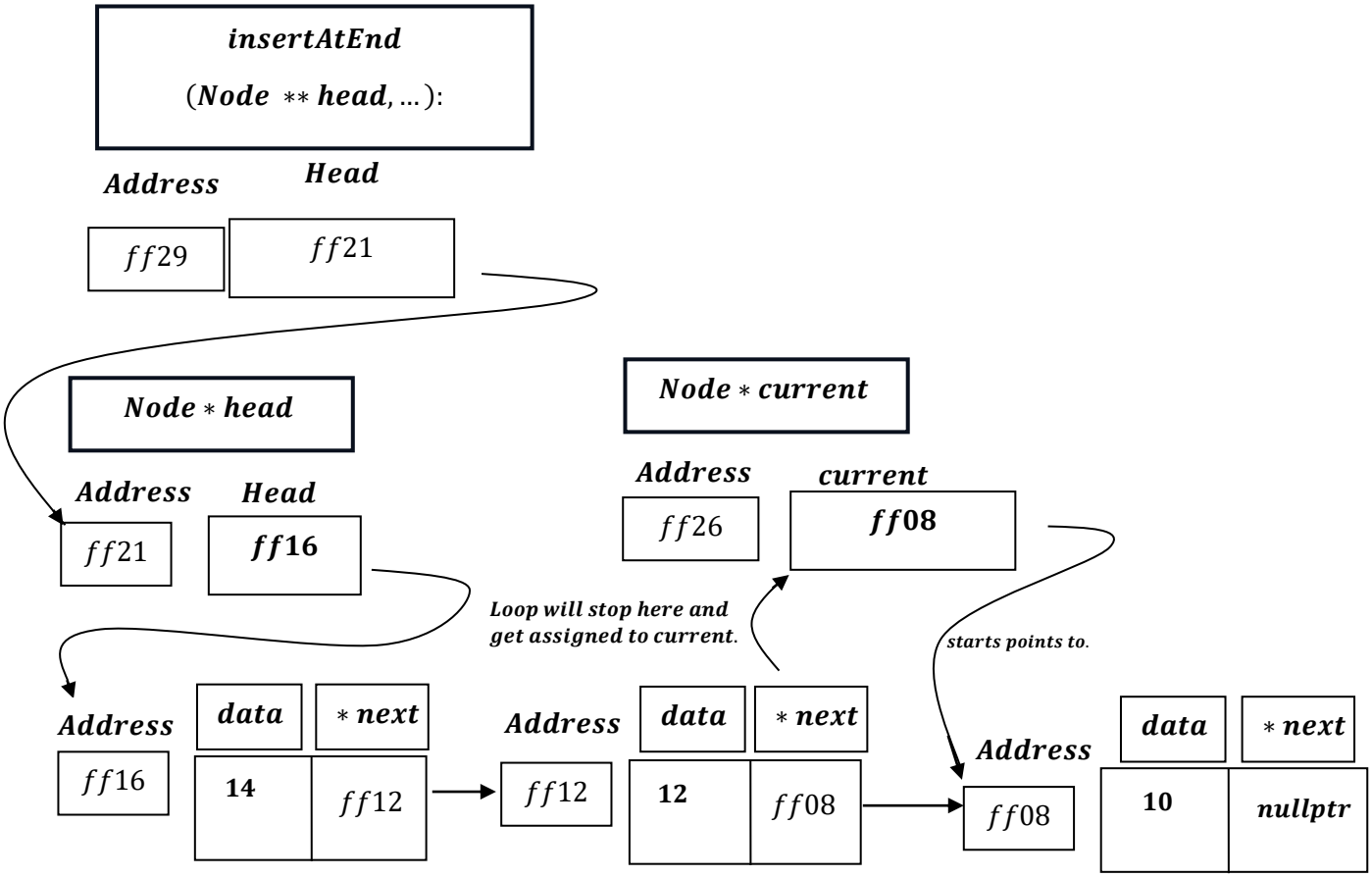
Now, one may ask generating a temporary pointer variable named current , while we can move with head pointer variable of Node structure?

Ans: It is simple , head must point to starting node's address and if we move through head it point to different address and through this process , our linked list will be tampered , so to remain the pointer head unaltered, we have taken current pointer variable.

Now, coming back to the program:

```
    while (current->next != nullptr)
      {
          current = current->next;
      }
```

Therefore , till the nullptr arrives , current pointer address values will get assigned and updated .



$Hence\ , during\ the\ loop\ terminates\ , current\ will\ be\ at$:

$1st\ \ time\ loop\ runs\ then\ , current = current \rightarrow next = ff08.$

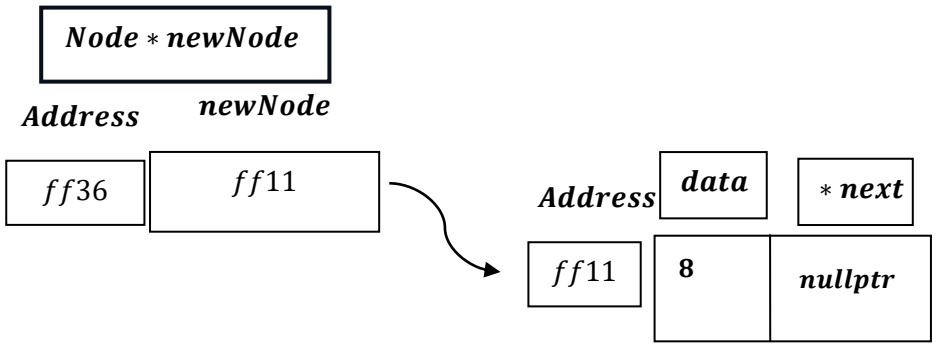
$2nd\ time\ , current = current \rightarrow next = nullptr, hence\ loop\ terminates.$

$Hence, loop\ runs\ only\ one\ time, when\ number\ of\ node\ is\ 2, i.e.\ loop\ runs\ n - 1\ time.$

$Now, we\ have\ newNode$:

```
    Node *newNode = (Node *)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = nullptr;
```

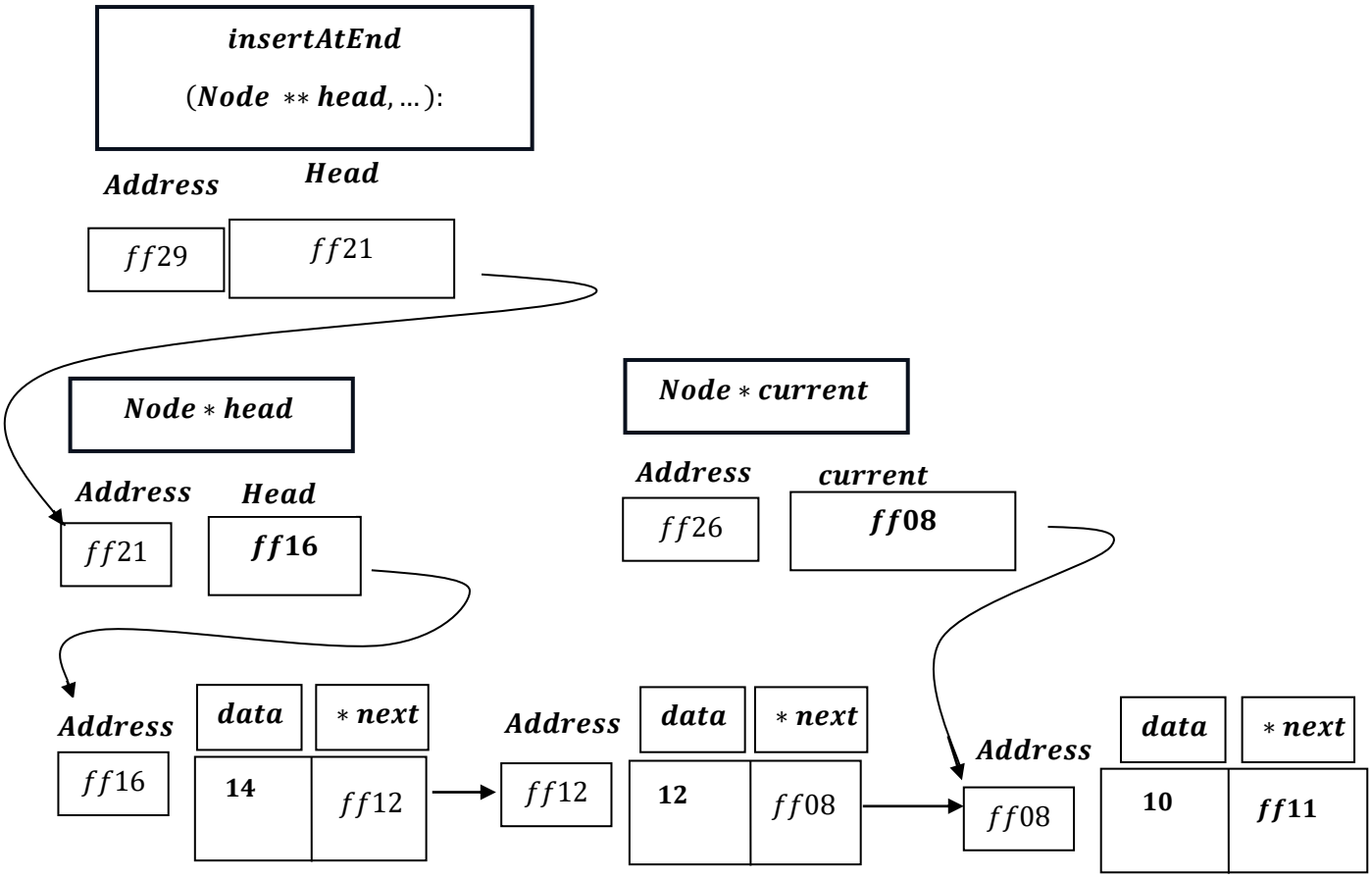
$And\ our\ data\ is$: 8 , $hence$:



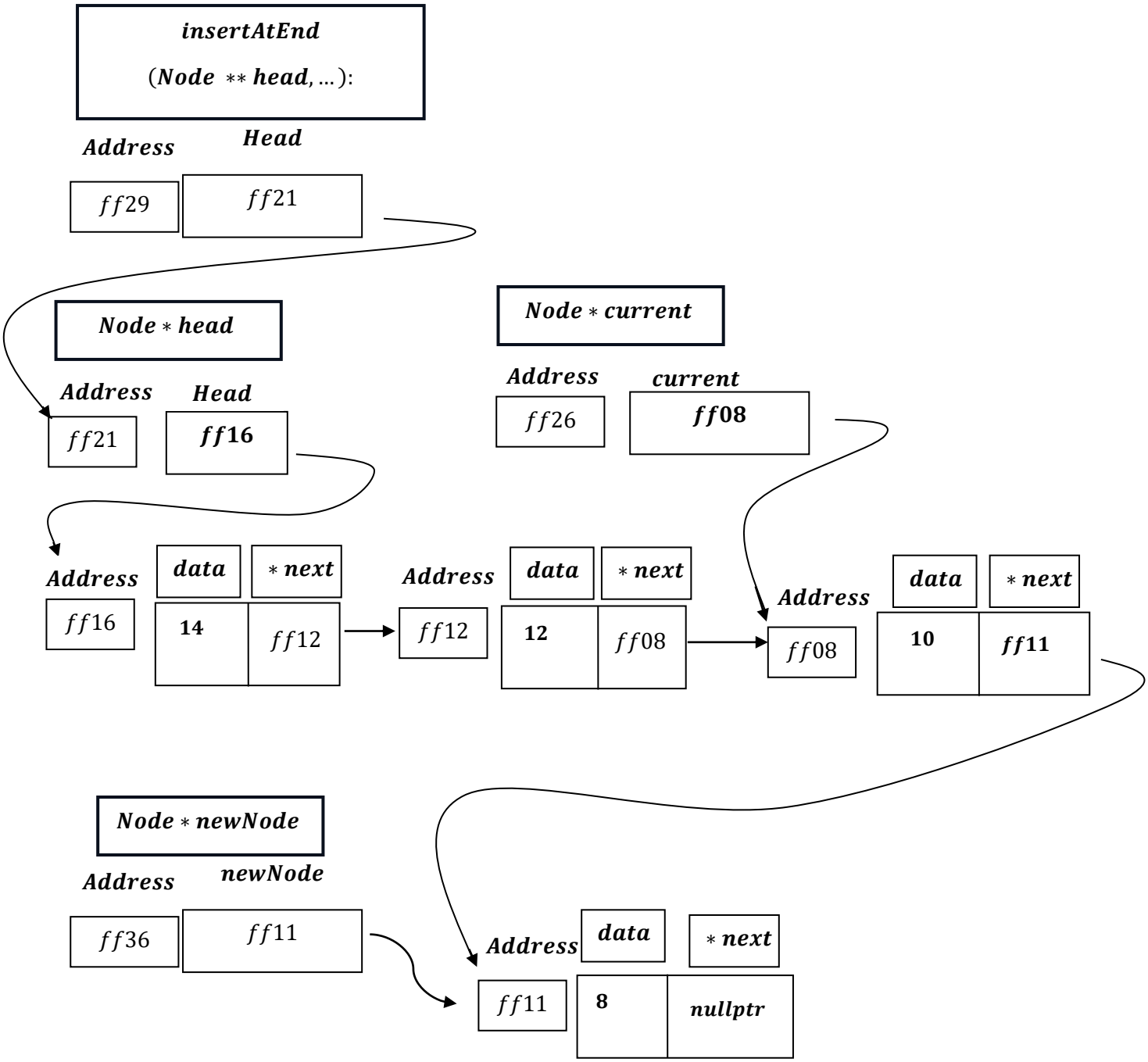
$Thereafter,$

```
    else
    {
        …..
        current->next = newNode;
    }
```

$The\ next\ pointer\ which\ is\ carrying\ nullptr\ assigned\ to\ value\ that\ newNode\ pointer\ is\ carrying.$

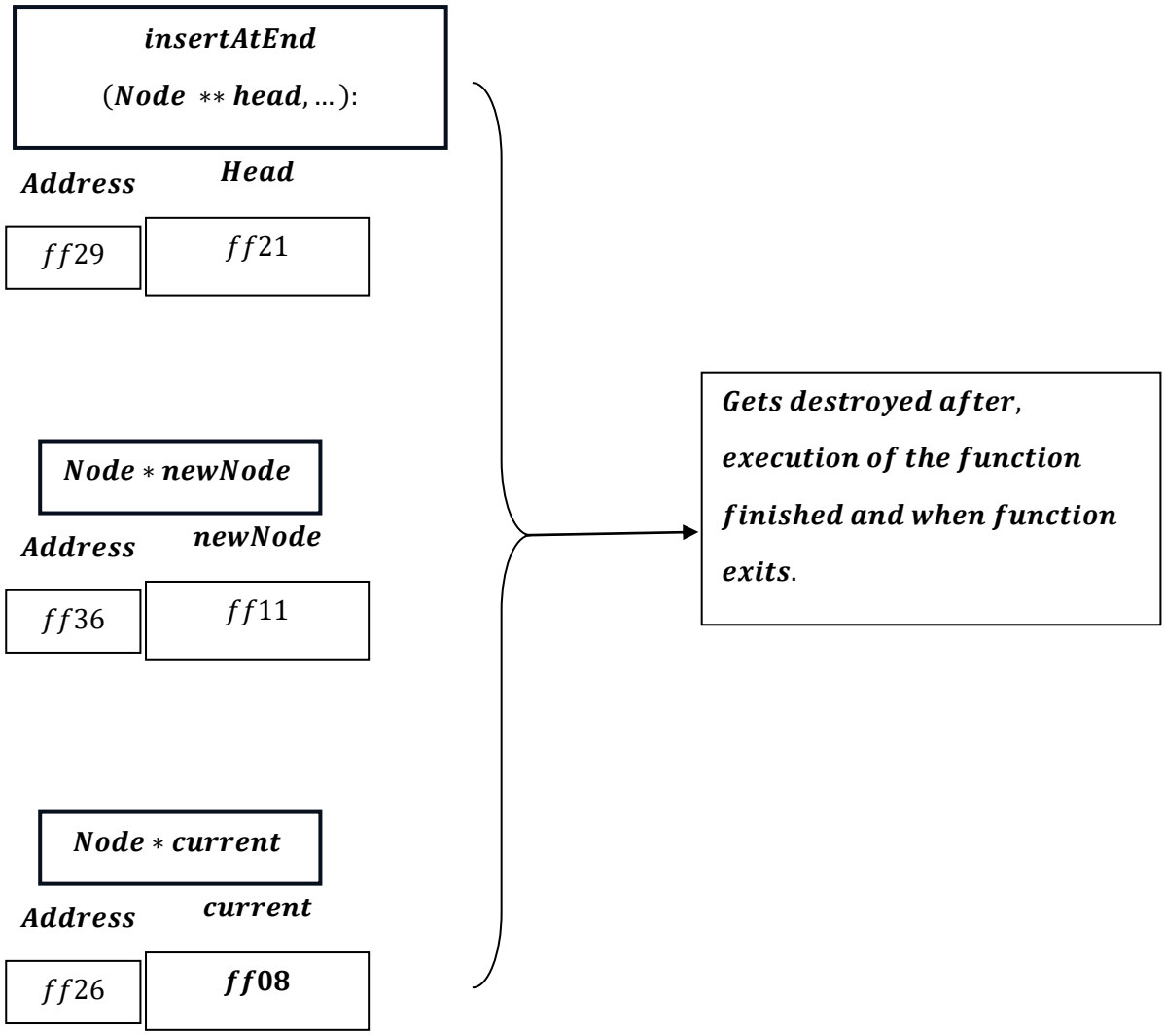


Now next pointer will point to the address, which was assigned to it.

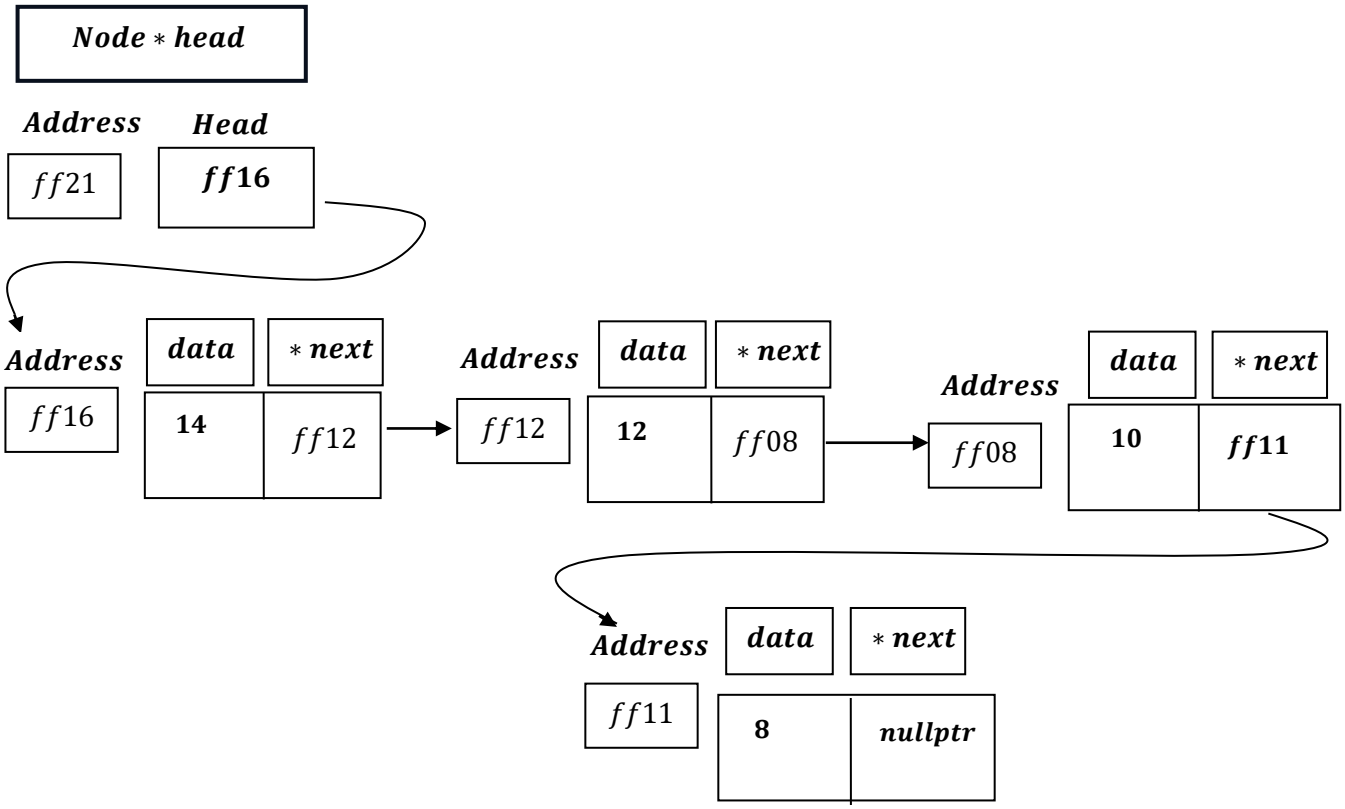


*Now, as Node * newNode , Node * current and Node ** head are temporary variable, therefore gets destroyed after function execution get finished .*

i.e.



Hence we are left with the linked list:



STEP BY STEP PROCESS

→ A double pointer to the head of the list (Node ** head).

→ An integer value (int data) that needs to be inserted at the end of the list.

→ Allocate memory for a new node.

→ Check if the memory allocation was successful. If not, print an error message and return from the function.

→ Set the data field of the new node to the value passed as a parameter.

→ Set the next pointer of the new node to nullptr because this node will be the last node in the list.

→ If the list is empty (i.e., * head is nullptr), set the head of the list to point to the new node.

→ If the list is not empty, initialize a pointer current to point to the head of the list.

→ Traverse the list until you reach the last node (i.e., a node whose next pointer is nullptr).

→ Set the next pointer of the last node to point to the new node, effectively adding the new node to the end of the list.

Time Complexity of Insert At End Function Module

```
void insertAtEnd(Node **head, int data)
{
    Node *newNode = (Node *)malloc(sizeof(Node));   → O(1)
    if (newNode == nullptr)   → O(1)
    {
        cout << "Error: Memory allocation failed." << endl;   → O(1)
        return;   → O(1)
    }
    newNode->data = data;   → O(1)
    newNode->next = nullptr;   → O(1)


    if (*head == nullptr)   → O(1)
    {
        *head = newNode;   → O(1)
    }
    else
    {
        Node *current = *head;   → O(1)
        while (current->next != nullptr)   → runs `n − 1` times.
        {
            current = current->next;   → 1 + 1+..(n − 1) times = O(n)
        }
        current->next = newNode;   → O(1)
    }
}
```

→ *The function begins by allocating memory for a new node using malloc. This operation takes constant time, typically* $O(1)$, *regardless of the size of the list.*

→ *Now checking newNode is assigned to NullPtr i.e. if the allocation was successful. If not, it prints an error message and returns. This step*

$also\ takes\ constant\ time, O(1).$

$\rightarrow If\ memory\ allocation\ is\ successful, the\ function\ initializes\ the\ new\ node's\ data\ and\ next\ pointer. Both\ of\ these\ operations\ take\ constant\ time, O(1),$
$as\ they\ only\ involve\ assigning\ values\ to\ the\ node's\ fields.$

$\rightarrow The\ function\ checks\ if\ the\ head\ of\ the\ list\ is\ null, indicating\ an\ empty\ list. If\ the\ list\ is\ empty, the\ new\ node\ becomes\ the\ head. This\ step\ takes\ constant\ time, O(1).$

$\rightarrow If\ the\ list\ is\ not\ empty, the\ function\ uses\ a\ loop\ to\ traverse\ the\ list\ and\ find\ the\ last\ node. In\ each\ iteration\ of\ the\ loop, the\ current\ pointer\ is\ moved\ to\ the\ next\ node. The\ number\ of\ iterations\ required\ is\ equal\ to\ the\ one\ less\ than\ number\ of\ nodes\ in\ the\ list, n\ i.e. n-1. Therefore, the\ time\ complexity\ of\ this\ step\ is\ O(n-1) = O(n) - O(1) = O(n).$

$\rightarrow Once\ the\ loop\ reaches\ the\ end\ of\ the\ list, the\ new\ node\ is\ inserted\ by\ setting\ the\ next\ pointer\ of\ the\ last\ node\ to\ point\ to\ the\ new\ node. This\ operation\ takes\ constant\ time, O(1).$

$\therefore When\ list\ is\ Empty\ overall\ \ time\ complexity\ is:$

$O(1) + O(1) + O(1) + O(1) + O(1) + O(1) = O(1), constant\ time\ complexity.$

$\therefore When\ list\ is\ not\ Empty\ overall\ \ time\ complexity\ is:$

$O(1) + O(1) + O(1) + O(1) + O(n) + O(1) = O(n).$

Singly Linked List	Condition	Time Complexity
InsertAtEnd	1. List is Empty	$O(1)$
	2. List is not Empty	$O(n)$

We can say , if List is Empty , we get our best case scenario, and when list is not empty , we get our worst case scenario. But though best case and worst case scenarios are present as, always we can determine where the insertion to be made , hence we can say average case is our worst case here i.e. always the entire list is traversed i.e. $O(n) = \Theta(n)$.

In the case of insertAtEnd, the insertion always happens at the end, regardless of the current state of the list. This means the execution time is solely dependent on the length of the list. If the list is empty, the execution time is constant (best case). If the list is not empty, the execution time is proportional to the length of the list (worst case). Since the insertion position is predetermined, there's no element of randomness or varying input data to consider. Therefore, the concept of an average case, where we average the execution time across different input possibilities, doesn't strictly apply here. Hence insertAtEnd doesn't have a true average case.

Note , by relation: $\Omega \leq \Theta \leq O$, we get:

$\Rightarrow c_1 \times g(n) \leq f(n) \leq c_2 \times g(n)$

$\Rightarrow c_1 \times 1 \leq N \leq c_2 \times N$

Singly Linked List	***Cases***	***Time Complexity***
InsertAt End	**1.** ***Best Case***	$\boldsymbol{\Omega(1)}$
	2. ***Worst Case***	$\boldsymbol{O(n)}$
	3. ***Average Case***	$\boldsymbol{\Theta(n)}$

****